



UFRJ

Universidade Federal do Rio de Janeiro

CCMN – IC / Instituto de Ciência da Computação

Miguel Angelo Gonzaga Marques - 122138896

Compressão de Imagens via Transformada de Wavelet e JPEG 2000

Rio de Janeiro

2026

1. Introdução

A transformada de *Wavelet* ("ondinha" em tradução literal) é uma técnica de análise de sinais, usada em várias áreas: compressão de imagens e vídeo, remoção de ruído, análise de áudio, e até na detecção de ondas gravitacionais.

A ideia principal desse trabalho foi entender a transformada de Wavelet do ponto de vista da Álgebra Linear, construindo ela do zero (começando do caso mais simples possível) até chegar em um pipeline de compressão de imagens no estilo do padrão JPEG 2000. O método gira em torno de uma pergunta simples: se uma imagem é só um vetor gigante de números, existe uma base melhor para escrever esse vetor? Veremos que sim, e que isso permite jogar fora mais de 95% dos números e ainda reconstruir uma imagem visualmente quase idêntica à original.

Todo o código desenvolvido está disponível em um repositório do GitHub, na forma de um notebook que constrói cada etapa descrita ao longo do relatório: [Repositório do GitHub](#)

2. Sinais, imagens e mudanças de base

2.1 Sinais e imagens são vetores

Um sinal digital com n amostras é simplesmente um vetor $x \in \mathbb{R}^n$. Uma imagem em escala de cinza de 512×512 pixels é uma matriz, mas também pode ser vista como um vetor de 262.144 dimensões (*data flattening*). Nas nossas imagens, o valor de cada pixel será um número decimal $0 \leq x \leq 1$ onde $x = 0$ representa a cor preta e $x = 1$ representa a cor branca.

Quando armazenamos uma imagem "pura", estamos representando esse vetor na base canônica: cada coeficiente diz o valor de um pixel específico. O problema é que essa base é péssima para compressão, já que cada coeficiente carrega pouquíssima informação sozinho, e nenhum deles é

"menos importante" que o outro. Se jogarmos fora metade dos coeficientes da base canônica, jogamos fora metade dos pixels da imagem.

2.2 Mudança de base: o coração de tudo

A ideia central de qualquer transformada usada em compressão (Fourier, DCT, PCA, Wavelet) é trocar de base. Queremos encontrar uma base ortonormal $\{w_1, w_2, \dots, w_n\}$ de $\mathbb{R}^n$ tal que, ao escrever nosso sinal como combinação linear

$$x = c_1 w_1 + c_2 w_2 + \dots + c_n w_n,$$

a maioria dos coeficientes c_i é quase zero. Se conseguirmos isso, podemos guardar só os poucos coeficientes grandes e jogar o resto fora, e isso é o que chamamos de compressão. As wavelets serão a base que concentra a energia do sinal (a energia de um sinal é definida como $\|x\|^2$, a soma dos quadrados das suas amostras).

Relembrando que, em uma base ortonormal, os coeficientes são calculados por produtos internos, $c_i = \langle w_i, x \rangle$, e vale a identidade de Parseval, $\|x\|^2 = c_1^2 + \dots + c_n^2$ (expandindo $\langle x, x \rangle$ na base, os produtos cruzados somem por ortogonalidade). Ou seja, a energia do sinal é a mesma em qualquer base ortonormal, só muda como ela está distribuída entre os coeficientes.

2.3 A limitação de Fourier: falta de localização

A transformada de Fourier decompõe o sinal em senos e cossenos. Ela é ótima para sinais estáticos (que se comportam igual o tempo todo), mas tem uma limitação intrínseca: cada seno ocupa o sinal inteiro. Se a imagem tem uma borda bem marcada em um canto, essa borda "contamina" os coeficientes de todas as frequências, e o coeficiente de uma frequência não diz *onde* aquela frequência acontece.

Imagens reais são cheias de eventos específicos, como bordas, texturas e objetos. O que queremos é uma base cujos vetores sejam localizados ao mesmo tempo em posição e em frequência; vetores que respondem só a "variações rápidas aqui" ou "comportamento médio ali". É exatamente isso que uma wavelet é: uma "ondinha" (daí vem o nome), que analisa o sinal em diferentes posições e diferentes escalas.

3. A wavelet de Haar

3.1 Médias e diferenças

A transformada de Haar (Alfréd Haar, 1909, antes até do termo *wavelet* existir) é a wavelet mais simples possível: pegue o sinal de dois em dois números e troque cada par pela sua média e pela sua diferença.

Se o par é (a, b) :

$$m\acute{e}dia \sim \frac{a+b}{2}, \quad diferen\c{c}a \sim \frac{a-b}{2}$$

Com a média e a diferença conseguimos recuperar o par exato ($a = m\acute{e}dia + diferen\c{c}a$ e $b = m\acute{e}dia - diferen\c{c}a$), então nenhuma informação é perdida, só armazenada de forma diferente.

- As médias formam a **aproximação**: uma versão do sinal com metade da resolução.
- As diferenças formam o **detalhe**: o que falta para reconstruir a resolução original a partir desse "resumo".

Isso ajuda pois se o sinal é suave, $a \approx b$ em quase todo par, e as diferenças ficam quase todas próximas de zero. A informação se concentrou nas médias.

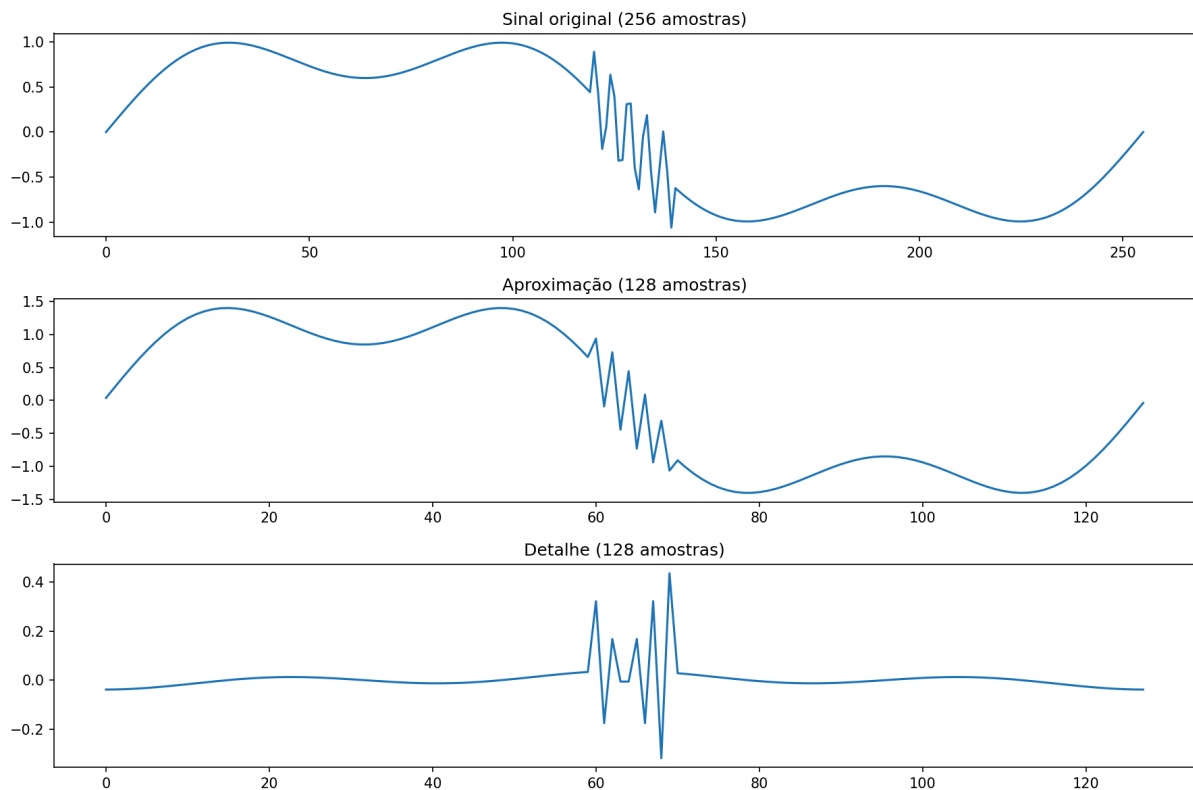


Imagem 1 - Um passo da transformada de Haar em um sinal de 256 amostras: a aproximação guarda o formato geral, e o detalhe só "acorda" na região de alta frequência inserida no meio do sinal.

Como podemos ver na Imagem 1, o detalhe é praticamente nulo nas regiões suaves do sinal, e concentra toda a sua atividade exatamente onde tem uma rajada de alta frequência. A localização que faltava em Fourier existe aqui, com cada coeficiente de detalhe olhando para um par específico de vizinhos.

3.2 A normalização por $\sqrt{2}$

Na implementação, não usamos a média e a diferença padrão, e sim as versões normalizadas:

$$s_i = \frac{x_{2i} + x_{2i+1}}{\sqrt{2}}, \quad d_i = \frac{x_{2i} - x_{2i+1}}{\sqrt{2}}$$

Isso porque queremos que a transformação preserve a norma (energia) do sinal. Observando com um par (a, b) :

$$s_i^2 + d_i^2 = \frac{(a+b)^2 + (a-b)^2}{2} = \frac{2a^2 + 2b^2}{2} = a^2 + b^2$$

Somando sobre todos os pares, obtemos $\|x\|^2 = \|s\|^2 + \|d\|^2$: a energia total é redistribuída entre aproximação e detalhe (a identidade de Parseval da seção 2.2 em miniatura). Fazer isso é o que torna a matriz da transformada ortogonal (seção 3.3), e é o que permite comparar as magnitudes dos coeficientes de forma direta na hora de comprimir.

Observe que a inversa usa exatamente as mesmas fórmulas:

$$x_{2i} = \frac{s_i + d_i}{\sqrt{2}}, \quad x_{2i+1} = \frac{s_i - d_i}{\sqrt{2}}$$

3.3 A matriz de Haar e suas consequências

O passo de Haar é uma operação linear em x , então existe uma matriz W tal que o vetor transformado é Wx . Para $n = 8$, organizando com aproximação em cima e detalhe embaixo:

$$W = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & & & & & & \\ & & 1 & 1 & & & & \\ & & & & 1 & 1 & & \\ & & & & & & 1 & 1 \\ 1 & -1 & & & & & & \\ & & 1 & -1 & & & & \\ & & & & 1 & -1 & & \\ & & & & & & 1 & -1 \end{pmatrix}$$

As 4 primeiras linhas calculam as médias normalizadas (vetores de "escala") e as 4 últimas calculam as diferenças (vetores "wavelet"). Repare que cada linha é um vetor curto e localizado: só duas entradas não nulas, cada uma para um pedaço do sinal. Isso é a localização da seção 2.3, visível na matriz.

Teorema. A matriz de Haar W é ortogonal, isto é, $W^T W = I$ (equivalentemente, $W^{-1} = W^T$).

Demonstração. Basta verificar que as linhas de W formam um conjunto ortonormal. Temos três casos:

1. *Norma de cada linha:* toda linha tem exatamente duas entradas não nulas, iguais a $\pm \frac{1}{\sqrt{2}}$, então sua norma é $\sqrt{\frac{1}{2} + \frac{1}{2}} = 1$.
2. *Linha de média e linha de diferença do mesmo par:* o produto interno é $\frac{1}{2} - \frac{1}{2} = 0$.
3. *Linhas de pares diferentes:* as entradas não nulas ocupam colunas disjuntas, então o produto interno é 0.

Como as n linhas são ortonormais, $W^T W = I$; e como W é quadrada, isso implica que W é invertível com $W^{-1} = W^T$. ■

Esse teorema simples tem consequências (úteis):

1. **A inversa é de graça.** Reconstruir o sinal é só multiplicar pela transposta: $x = W^T(Wx)$.
2. **A transformada é uma mudança de base ortonormal.** Os coeficientes Wx são as coordenadas de x na base formada pelas linhas de W . Geometricamente é só uma "rotação" do espaço, o vetor é o mesmo, o que muda é a régua que usamos para descrevê-lo.
3. **Preservação de energia:** $\|Wx\|^2 = x^T W^T W x = \|x\|^2$.
4. **O erro no domínio transformado é igual ao erro na imagem.** Essa é a consequência mais importante para compressão: se $c = Wx$ e produzimos uma versão corrompida $\tilde{c} = c + e$ (por exemplo, zerando alguns coeficientes), o erro na reconstrução é

$\|W^T \tilde{c} - W^T c\| = \|W^T e\| = \|e\|$, porque W^T também é ortogonal. Ou seja, o dano que causamos na imagem é exatamente a soma dos quadrados dos coeficientes que zeramos.

3.4 Custo computacional

Multiplicar por W custaria $O(n^2)$ operações, mas a matriz é muito esparsa: implementando o passo direto nas fatias pares e ímpares do sinal, o custo é $O(n)$, e a transformada multinível completa custa $n + \frac{n}{2} + \frac{n}{4} + \dots < 2n = O(n)$, sendo mais rápida que a FFT, que é $O(n \log n)$.

3.5 Decomposição multinível e análise multirresolução

Depois de um passo, temos uma aproximação e um detalhe, cada um com $\frac{n}{2}$ amostras. Mas a aproximação ainda é um sinal, então podemos transformá-la repetidas vezes. Repetindo L vezes, o sinal vira:

$$[a^{(L)}, d^{(L)}, d^{(L-1)}, \dots, d^{(1)}]$$

O total de coeficientes continua sendo n , continuamos sem perder nada, e cada nível é uma transformação ortogonal, então tudo da seção 3.3 continua valendo para a transformada composta. $d^{(1)}$ captura variações na escala mais fina (entre pixels vizinhos), $d^{(2)}$ numa escala duas vezes maior, e assim por diante; $a^{(L)}$ é um esboço bem grosseiro do sinal. É uma decomposição do sinal por escalas, e por isso o nome análise multirresolução (*multiresolution analysis*, MRA. Stéphane Mallat e Yves Meyer, 1989).

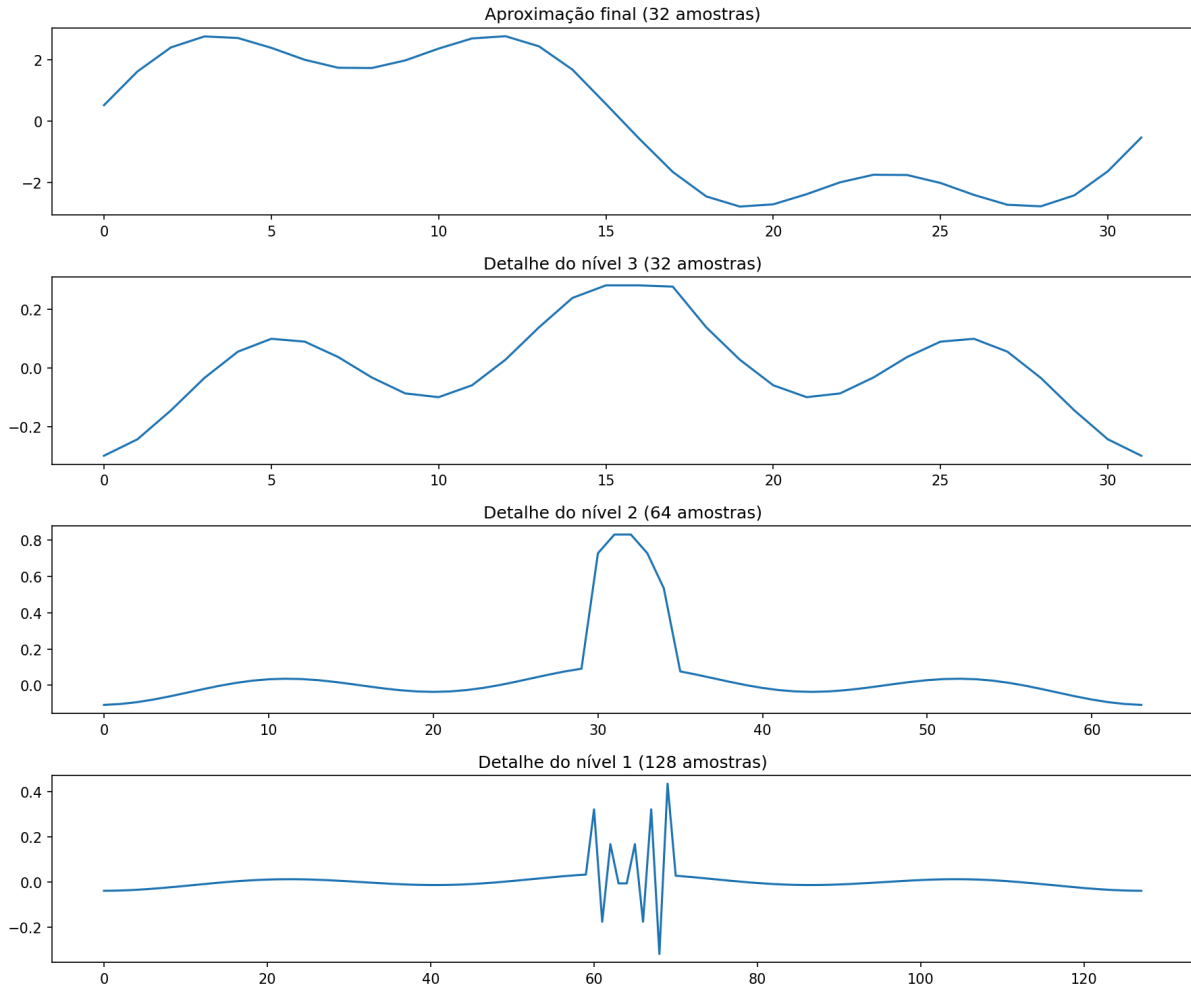


Imagem 2 - Decomposição de 3 níveis do mesmo sinal: uma aproximação de 32 amostras e os detalhes de cada escala.

Formalmente, a MRA organiza tudo isso em subespaços encaixados $V_L \subset \dots \subset V_1 \subset V_0$, onde cada V_j contém os sinais representáveis na resolução 2^{-j} , e o detalhe de nível j vive no complemento ortogonal W_j definido por $V_{j-1} = V_j \oplus W_j$ (não confundir com a matriz W): "sinal na resolução fina = sinal na resolução grossa + o que falta para refinar". Na linguagem de funções, V_j é gerado por translações da função de escala φ (para a Haar, o pulso retangular) e W_j por translações e dilatações da wavelet mãe ψ : todas as wavelets da família são a mesma "ondinha", esticada e deslocada, $\psi_{j,k}(t) = 2^{-\frac{j}{2}} \psi(2^{-j}t - k)$.

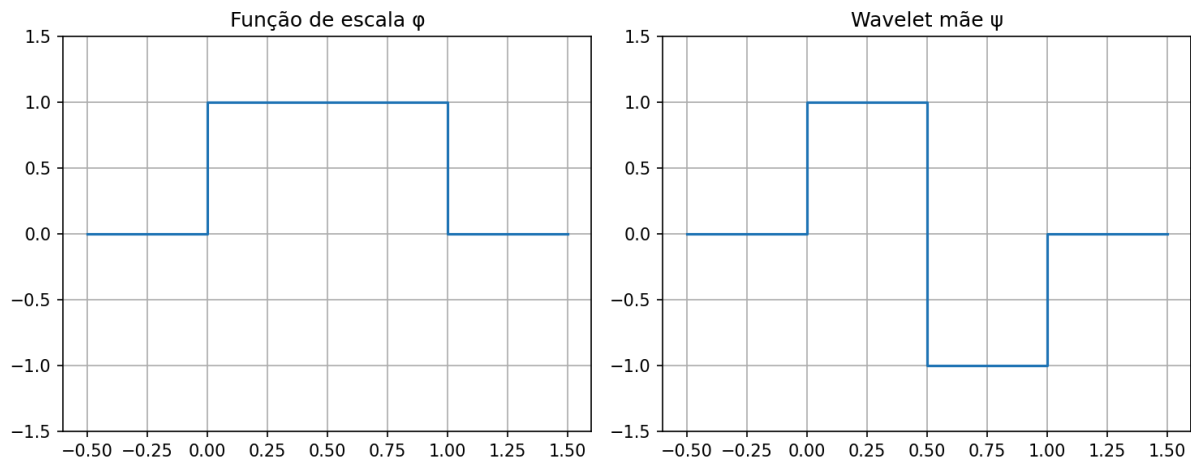


Imagem 3 - A função de escala φ e a wavelet mãe ψ de Haar. A ψ é literalmente a "ondinha" mais simples que existe: sobe uma vez, desce uma vez.

4. A transformada em imagens (2D)

4.1 Transformada separável

Para levar a transformada para 2D usamos o truque das transformadas separáveis: aplicá-la separadamente por dimensão. Primeiro transformamos cada linha da imagem, depois cada coluna desse resultado.

Em termos de Álgebra Linear, se A é a imagem $n \times n$: transformar as linhas é calcular AW^T , e transformar as colunas do resultado é multiplicar por W à esquerda. A transformada 2D completa é:

$$B = WAW^T$$

Teorema. A transformada 2D é perfeitamente invertível, com $A = W^T B W$, e também preserva energia: $\|B\|_F = \|A\|_F$ (norma de Frobenius).

Demonstração. Para a inversa, basta usar a ortogonalidade de W duas vezes:

$$W^T B W = W^T (W A W^T) W = (W^T W) A (W^T W) = A$$

Para a energia, usamos que $\|M\|_F^2 = \text{tr}(M^T M)$ e a propriedade cíclica do traço:

$$\|B\|_F^2 = \text{tr}(W A^T W^T W A W^T) = \text{tr}(A^T A W^T W) = \text{tr}(A^T A) = \|A\|_F^2. \blacksquare$$

Existe uma interpretação: escrever $B = W A W^T$ é a mesma coisa que expressar a imagem numa base de n^2 produtos externos $w_i w_j^T$, como se fossem vários "azulejos" localizados em posição e escala nas duas direções. Essa base 2D também é ortonormal ($\langle w_i w_j^T, w_k w_l^T \rangle = \delta_{ij} \delta_{kl}$), então todas as propriedades da seção 3.3 continuam valendo em 2D.

4.2 As quatro sub-bandas

Um passo 2D produz quatro quadrantes, conforme a combinação de filtro (passa-baixa L = média, passa-alta H = diferença) aplicada em cada direção:

Quadrante	Filtro	O que contém
LL (topo-esquerda)	média nas duas direções	aproximação, em metade da resolução
HL (topo-direita)	diferença na horizontal, média na vertical	bordas verticais

LH (baixo-esquerda)	média na horizontal, diferença na vertical	bordas horizontais
HH (baixo-direita)	diferença nas duas	cantos e detalhes diagonais

É importante tomar cuidado: HL captura variação horizontal, que visualmente corresponde a bordas verticais (uma linha vertical é justamente onde o valor muda quando andamos na horizontal).

LL (aproximação)



HL (detalhes verticais)



LH (detalhes horizontais)



HH (detalhes diagonais)



Imagem 4 - As quatro sub-bandas de um passo da transformada 2D na imagem de teste. Repare como HL destaca o tripé da câmera (bordas verticais) e LH destaca o horizonte (bordas horizontais).

4.3 A pirâmide multinível

Assim como no caso 1D, iteramos o passo 2D somente na sub-banda LL. O resultado é a estrutura em pirâmide característica: um LL pequeno no canto (uma "miniatura" da imagem) cercado por camadas de detalhes cada vez mais finos.

Decomposição com 3 níveis



Imagem 5 - Decomposição 2D com 3 níveis (as magnitudes estão em escala logarítmica, senão os detalhes seriam invisíveis de tão pequenos, o que aliás é ótimo sinal para a compressão).

5. Compressão por limiarização

5.1 Concentração de energia

Por que essa base é boa para compressão? A Imagem 6 mostra as magnitudes dos coeficientes da decomposição, ordenadas da maior para a menor, em escala logarítmica:

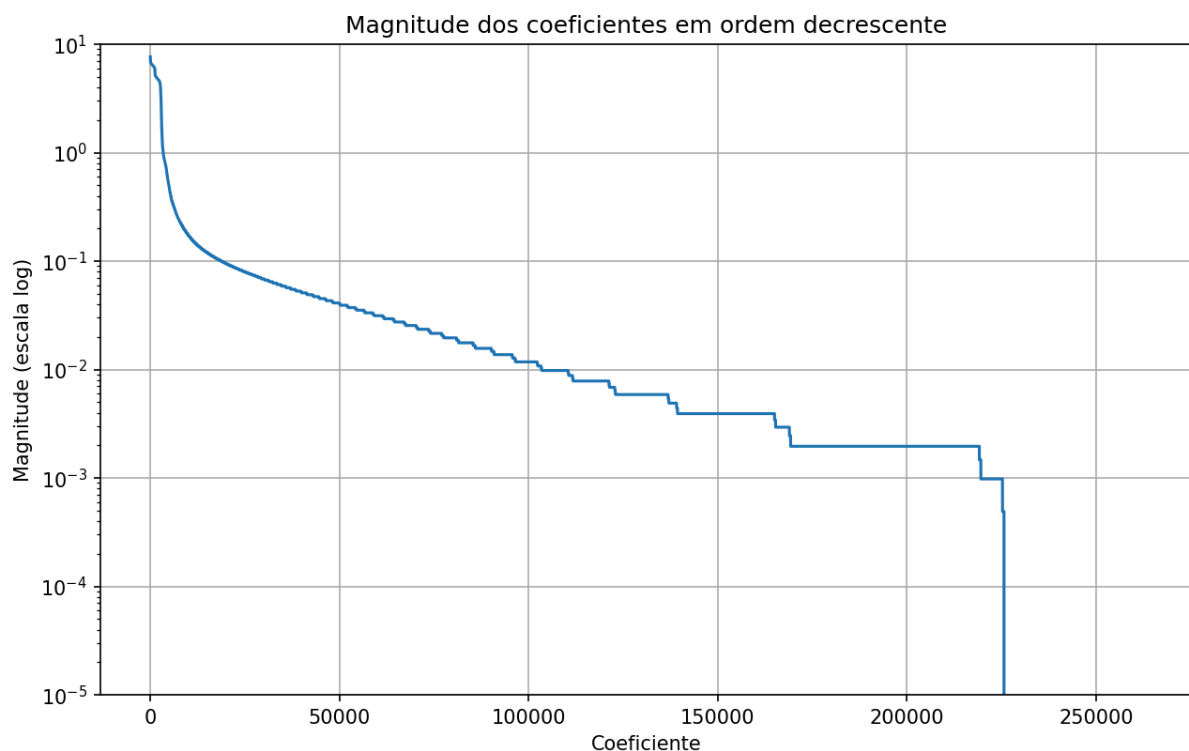


Imagem 6 - Magnitude dos coeficientes da transformada em ordem decrescente (escala log). Quase todos os coeficientes minúsculos.

Como podemos ver, uma quantidade muito pequena dos coeficientes carrega quase toda a energia da imagem. Fazendo a conta na nossa imagem de teste (com 4 níveis de decomposição), o 1% de

coeficientes de maior magnitude concentra 99,2% da energia total. Isso acontece porque imagens naturais são suaves por partes: grandes regiões quase constantes geram detalhes praticamente nulos, e coeficientes grandes só aparecem onde há bordas. E bordas são raras, elas são estruturas "1D" dentro de um objeto "2D".

5.2 Limiarização e o teorema da melhor aproximação

A estratégia de compressão mais simples possível é a limiarização (*thresholding*): fixar uma fração k dos coeficientes que queremos manter (por exemplo, 5%), encontrar o limiar de magnitude correspondente, e zerar todos os coeficientes abaixo dele.

Teorema (otimalidade da limiarização). Seja $c = Wx$ o vetor de coeficientes de x numa base ortonormal, e seja m o número de coeficientes que decidimos manter. Entre todas as escolhas possíveis de m coeficientes para manter (zerando os demais), manter os m de maior magnitude minimiza o erro quadrático $\|x - \tilde{x}\|^2$ da reconstrução.

Demonstração. Seja S o conjunto dos índices mantidos, com $|S| = m$, e $\tilde{c}$ o vetor com $\tilde{c}_i = c_i$ para $i \in S$ e $\tilde{c}_i = 0$ caso contrário. Pela consequência 4 da seção 3.3 (o erro no domínio transformado é igual ao erro na imagem):

$$\|x - \tilde{x}\|^2 = \|c - \tilde{c}\|^2 = \sum_{i \notin S} c_i^2$$

Ou seja, o erro é a soma dos quadrados dos coeficientes descartados. Essa soma é minimizada quando descartamos os $n - m$ coeficientes de menor magnitude, isto é, quando S contém os m maiores. ■

5.3 Medindo a qualidade: MSE e PSNR

Para comparar a imagem original x com a reconstruída $\tilde{x}$, usamos o erro quadrático médio (*MSE*) e o *Peak Signal-to-Noise Ratio* (razão sinal-ruído de pico, *PSNR*):

$$MSE = \frac{1}{n} \sum_i (x_i - \tilde{x}_i)^2, \quad PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

Como nossas imagens vivem em $[0, 1]$, $MAX = 1$ e o $PSNR$ vira simplesmente $-10 \log_{10}(MSE)$.

Na prática: acima de ~ 40 dB a compressão é visualmente impossível de diferenciar do original; 30-35 dB é uma qualidade boa; abaixo de ~ 25 dB a degradação começa a incomodar. Esses valores vêm de estudos empíricos de percepção, não têm nada a ver com Álgebra Linear (mas são a régua que todo mundo usa).

5.4 Resultados

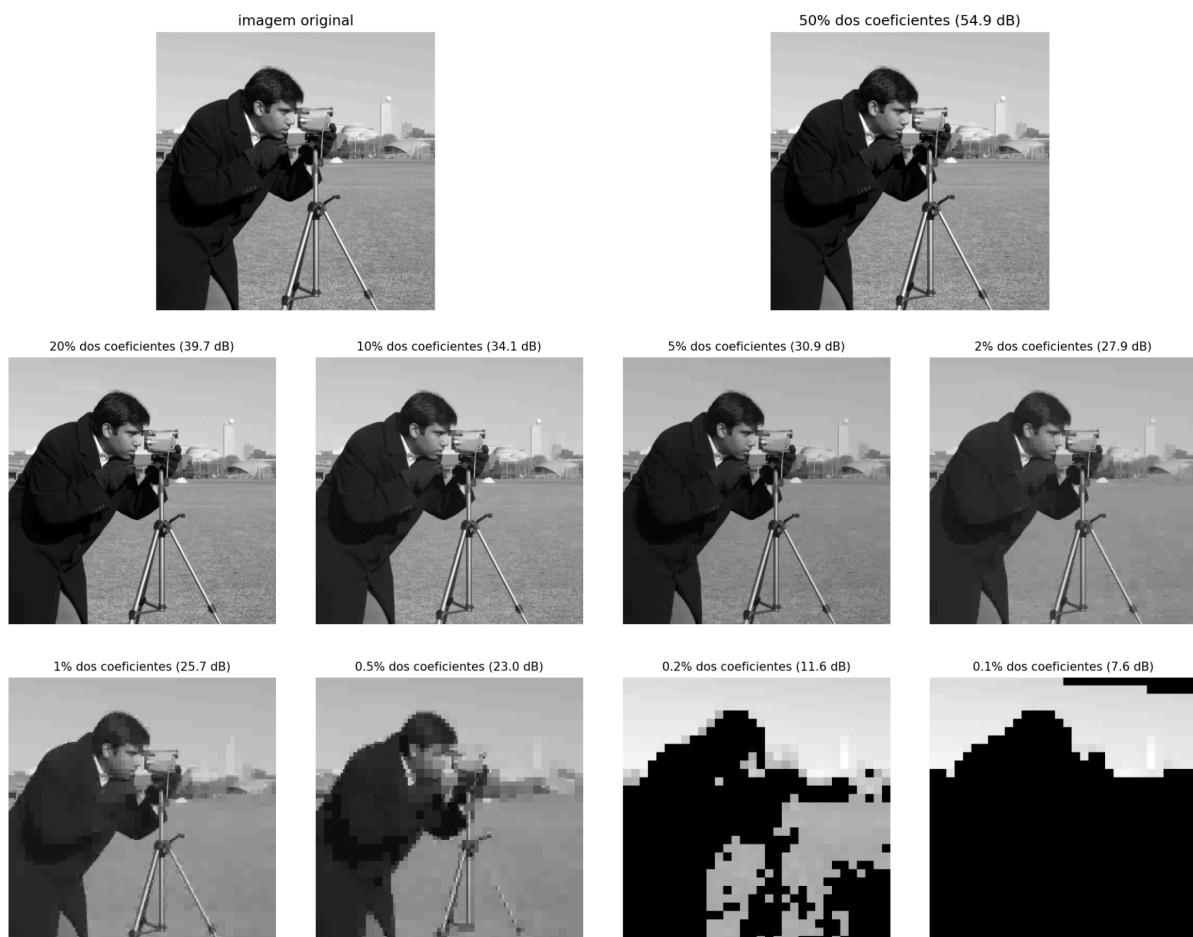


Imagem 7 - Compressão por limiarização mantendo de 50% até 0,1% dos coeficientes, com o PSNR de cada reconstrução.

Os resultados confirmam que mantendo 50% dos coeficientes temos 54,9 dB (perfeito para qualquer olho humano); com 10% ainda temos 34,1 dB, uma imagem visualmente boa; e mesmo com somente 2% dos coeficientes obtemos 27,9 dB, uma imagem completamente reconhecível.

Nos extremos, entretanto, acontece um colapso: de 0,2% para baixo a imagem desmorona completamente (11,6 dB). Com 4 níveis, a sub-banda LL tem $32 \times 32 = 1024$ coeficientes, o que dá 0,39% do total. Quando pedimos para manter só 0,2%, nem o LL inteiro sobrevive, e começamos a perder a estrutura básica da imagem, não só os detalhes. Por isso, os esquemas reais de compressão protegem esses coeficientes de aproximação.

6. A wavelet do JPEG 2000: a CDF 9/7 e o esquema lifting

A Haar é quase perfeita. Ela é ortogonal, rápida, e com inversa trivial. Mas o JPEG 2000 não usa ela, e essa seção explica o porquê.

6.1 O defeito da Haar: descontinuidade

A wavelet de Haar é descontínua (um degrau). Quando zeramos ou quantizamos muitos coeficientes, o erro que aparece na imagem tem o formato das funções da base, e degraus geram artefatos em blocos, muito parecidos com os do JPEG clássico. Uma wavelet suave produz erros suaves, e um borrão suave incomoda muito menos o olho humano que vários bloquinhos.

6.2 Momentos nulos

Uma wavelet Ψ tem p momentos nulos (*vanishing moments*) se $\int t^k \psi(t) dt = 0$ para $k = 0, 1, \dots, p - 1$. Na prática, isso significa que o filtro de detalhe zera qualquer polinômio de grau menor que p (a verificação é uma expansão direta pelo binômio de Newton). Pelo Teorema de Taylor, uma função suave é localmente bem aproximada por um polinômio, então o filtro de detalhe zera essa parte polinomial inteira e sobra só um resto minúsculo. Ou seja: mais momentos nulos $\Rightarrow$ detalhes ainda menores em regiões suaves $\Rightarrow$ mais esparsidade $\Rightarrow$ melhor compressão.

A Haar tem $p = 1$ (zera constantes). A CDF 9/7 tem $p = 4$, então, em vez de medir a "diferença do vizinho", ela mede o quanto o sinal foge de uma cúbica local. Para imagens suaves por partes, isso é uma diferença muito grande.

6.3 Simetria e biortogonalidade

Além de mais momentos nulos, queremos filtros simétricos. Eles não "deslocam" as bordas da imagem e permitem o tratamento de bordas por espelhamento (que veremos mais à frente). O problema é que Daubechies demonstrou que não existe wavelet real, ortogonal, de suporte finito e simétrica além da própria Haar. Então se quisermos simetria e suavidade, precisaremos abrir mão da ortogonalidade.

A solução são as wavelets biortogonais. São duas bases trabalhando em dupla, uma de análise $\{\tilde{w}_i\}$ (que decompõe o sinal) e uma de síntese $\{w_j\}$ (que reconstrói), satisfazendo a condição de dualidade $\langle \tilde{w}_i, w_j \rangle = \delta_{ij}$. Essa condição garante que a análise recupere exatamente os coeficientes da expansão na base de síntese, então a reconstrução perfeita continua valendo. O que perdemos é a preservação exata de energia, já que Parseval dependia da ortonormalidade. Na prática, a CDF 9/7 foi projetada para ser "quase ortogonal", e o argumento de otimalidade da limiarização continua valendo de forma aproximada. É um preço (pequeno) a se pagar por simetria + suavidade + 4 momentos nulos.

A CDF 9/7 (Cohen–Daubechies–Feauveau, 1992) usa filtro passa-baixa de análise com 9 coeficientes e de síntese com 7 (daí o nome). É a wavelet do modo com perdas do JPEG 2000.

6.4 O esquema lifting

Implementar a CDF 9/7 por convoluções diretas com os filtros de 9 e 7 coeficientes funciona, mas existe uma forma muito mais inteligente (e ~2x mais barata, feita *in-place*): o *lifting scheme* (esquema de elevação), proposto por Wim Sweldens nos anos 90. A ideia é construir a transformada em três movimentos:

1. **Split**: separa o sinal nas amostras pares $e_i = x_{2i}$ e ímpares $o_i = x_{2i+1}$.
2. **Predict (predição)**: prevê cada ímpar a partir dos pares vizinhos e guarda só o erro de predição: $o_i \leftarrow o_i - P(e)_i$. Se o sinal é suave, a predição é boa e o erro (o futuro detalhe) é pequeno.
3. **Update (atualização)**: corrige os pares com os detalhes, $e_i \leftarrow e_i + U(o)_i$, para que a sequência dos pares preserve as propriedades médias do sinal.

A Haar é o lifting mínimo: prever o ímpar como igual ao par e atualizar o par com metade do detalhe (sobra exatamente a média). A CDF 9/7 usa dois rounds de predição/atualização, mais uma escala final:

$$o += \alpha(e_{esq} + e_{dir}) \text{ (predição 1)}$$

$$e += \beta(o_{esq} + o_{dir}) \text{ (atualização 1)}$$

$$o += \gamma(e_{esq} + e_{dir}) \text{ (predição 2)}$$

$$e += \delta(o_{esq} + o_{dir}) \text{ (atualização 2)}$$

$$\text{aproximação} = \zeta e, \text{ detalhe} = \frac{o}{\zeta}$$

com as constantes definidas no padrão:

Constante	Valor	Papel
α	$-1,586134342$	predição 1
β	$-0,05298011854$	atualização 1
γ	$0,8829110762$	predição 2
δ	$0,4435068522$	atualização 2
ζ	$1,149604398$	normalização

Esses valores vêm da fatoração do banco de filtros 9/7 em passos de lifting. A constante ζ , em particular, é escolhida para que um sinal constante ganhe o mesmo fator $\sqrt{2}$ da Haar, deixando as duas wavelets na mesma régua de comparação.

O melhor do lifting é que a inversa é estrutural: cada passo soma em o algo que só depende de e (ou vice-versa), então para desfazer basta subtrair exatamente a mesma quantidade, aplicando os passos na ordem contrária com o sinal trocado ($o' = o + \alpha(e_{esq} + e_{dir}) \Rightarrow o = o' - \alpha(e_{esq} + e_{dir})$). Isso vale para qualquer valor das constantes, qualquer tratamento de borda, até se arredondarmos a cada passo para inteiros. É assim que o JPEG 2000 faz compressão sem perdas, com a wavelet 5/3 inteira.

O último problema é que o ímpar da última posição precisa de um vizinho que não existe. A solução do padrão é a extensão simétrica, que espelha o sinal na borda. Como os filtros são simétricos, o espelhamento não cria descontinuidade, e é por isso que a simetria da seção 6.3 importava.

6.5 Haar vs CDF 9/7 na prática

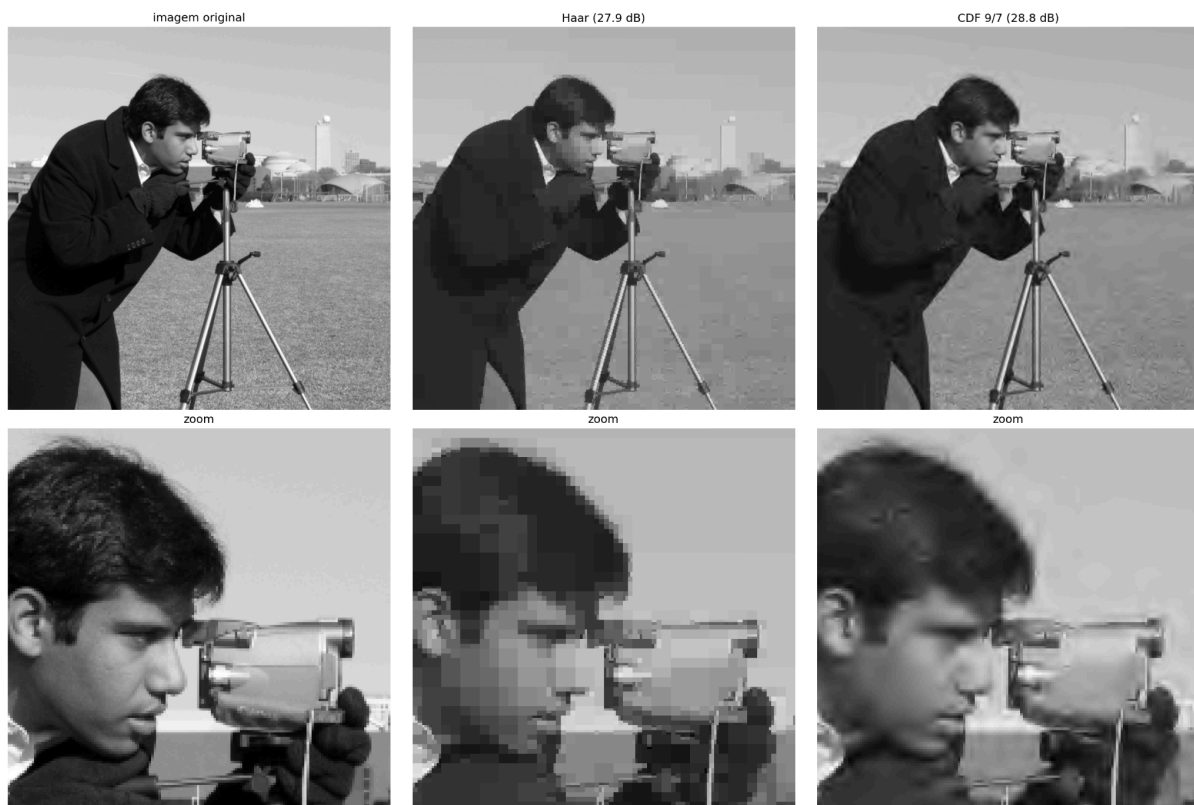


Imagem 8 - Compressão mantendo 2% dos coeficientes com as duas wavelets, com zoom no rosto. A Haar (27,9 dB) produz os blocos característicos; a CDF 9/7 (28,8 dB) troca eles por um borrão suave.

Como podemos ver, a diferença numérica parece pequena (0,9 dB), mas a diferença visual perceptível: no zoom, a reconstrução da Haar é feita de quadradinhos, enquanto a da CDF 9/7 é suave. Isso mostra uma coisa importante: o PSNR não pode ser levado em consideração sozinho, e o tipo de artefato importa tanto quanto a sua magnitude.

7. O padrão JPEG 2000

7.1 O pipeline completo

Com a transformada pronta, podemos descrever o pipeline do padrão de verdade (ISO/IEC 15444-1):

1. **Divisão em *tiles*** (opcional): a imagem pode ser dividida em blocos grandes e independentes para limitar o uso de memória e permitir decodificação regional; Diferente dos blocos 8×8 do JPEG clássico, os tiles são grandes e a transformada dentro deles é global, sem o efeito de blocagem.
2. **DC level shift**: subtrai o ponto médio do intervalo de valores (128 para 8 bits; 0,5 no nosso caso em $[0, 1]$) para centralizar os dados em zero.
3. **Transformada de cor** (para imagens coloridas): $RGB \rightarrow YCbCr$ no modo com perdas, ou uma versão inteira reversível no modo sem perdas. No nosso trabalho usamos somente escala de cinza.
4. **DWT 2D multinível**: normalmente de 4 a 6 níveis. CDF 9/7 no modo com perdas; LeGall 5/3 (lifting inteiro) no modo sem perdas.
5. **Quantização por sub-banda**: cada sub-banda b recebe um passo Δ_b , e cada coeficiente vira o inteiro $q = \text{sign}(c) \lfloor \frac{|c|}{\Delta_b} \rfloor$. Sub-bandas finas, que o olho percebe menos, recebem passos maiores. Este é o único lugar do pipeline onde obrigatoriamente se perde informação.
6. **EBCOT** (*Embedded Block Coding with Optimized Truncation*, de David Taubman): cada sub-banda é cortada em *code-blocks* codificados por planos de bits, com um codificador aritmético adaptativo. O resultado é um fluxo embutido: truncar em qualquer ponto dá a melhor imagem possível para aquele tamanho.
7. **Organização do *codestream***: camadas de qualidade e marcadores dão ao formato decodificação progressiva por qualidade ou resolução, regiões de interesse e acesso aleatório espacial.

7.2 Nossa implementação simplificada

Na nossa implementação, seguimos as etapas 2, 4 e 5 (DC level shift, DWT com a CDF 9/7 e quantização por sub-banda, com o passo dobrando a cada nível mais fino). A única etapa "simulada" é a 6: implementar o EBCOT completo fugiria do escopo, então estimamos o tamanho final usando a entropia de Shannon dos coeficientes quantizados:

$$H = - \sum_v p_v \log_2 p_v \text{ (bits por coeficiente)}$$

onde p_v é a frequência relativa do valor v . Pelo teorema da codificação de fonte de Shannon, H é um limite inferior para a média de bits por símbolo de qualquer codificador sem perdas, e os codificadores aritméticos modernos chegam muito perto desse limite; Portanto $H \times \frac{n}{8}$ bytes é uma estimativa justa do tamanho do arquivo final.



Imagem 9 - O pipeline completo na imagem de teste, com passos de quantização crescentes. Com taxa 7,5:1 temos 34,7 dB; até na taxa agressiva de 54,7:1 (26,4 dB) a imagem continua completamente reconhecível, sem nenhum bloco.

Para comparação entre as duas wavelets, medimos a curva de qualidade por taxa de compressão das duas no pipeline completo:

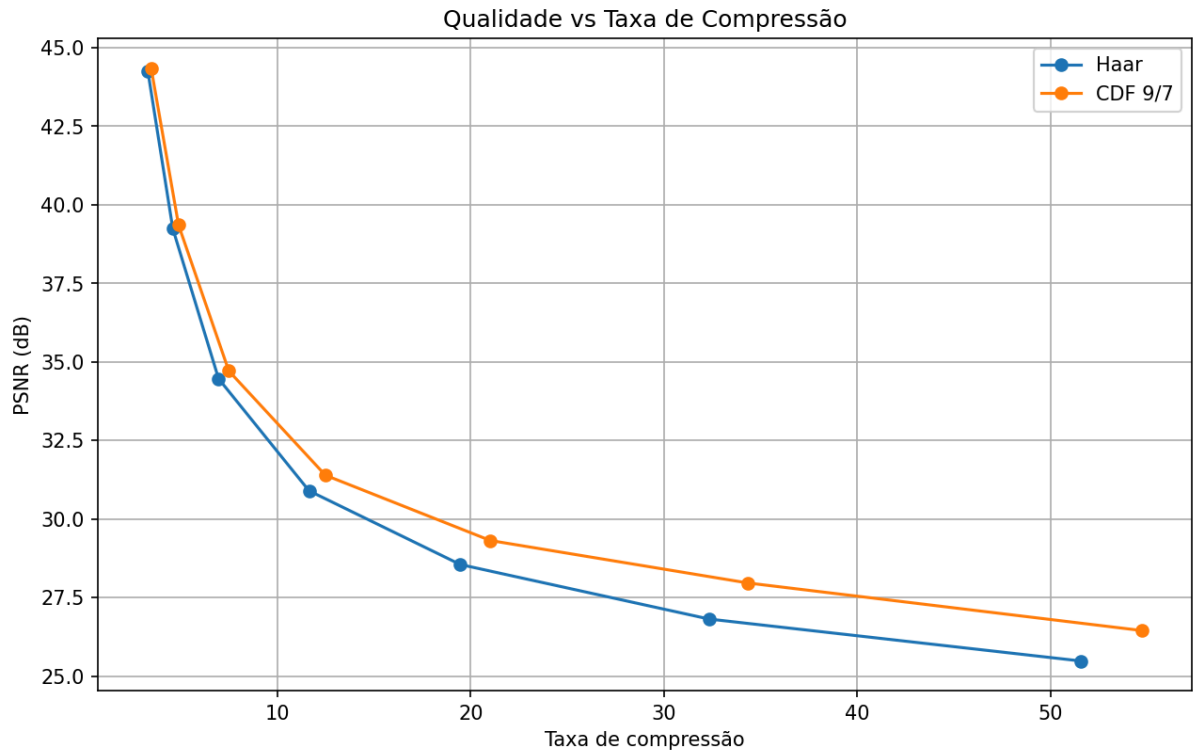


Imagem 10 - PSNR vs taxa de compressão para Haar e CDF 9/7 no mesmo pipeline. A CDF 9/7 vence em todas as taxas, e a vantagem cresce conforme a compressão fica mais agressiva.

Como podemos ver, a CDF 9/7 domina a Haar em toda a curva (e lembrando da Imagem 8, a vantagem visual é ainda maior do que os números sugerem). São os 4 momentos nulos e a suavidade que dão essa vantagem.

7.3 JPEG clássico vs JPEG 2000

	JPEG (1992)	JPEG 2000
Transformada	DCT em blocos 8×8	wavelet CDF 9/7, global (por tile)

Base	cosenos (localizados só por causa do bloco)	wavelets (localizadas por construção, multiescala)
Artefato típico	blocagem 8×8	borramento suave / <i>ringing</i> leve
Sem perdas	não (modo à parte, raro)	sim, no mesmo framework (5/3 inteira)
Multirresolução	limitada	nativa
Adoção	universal	nichos (cinema digital, imagens médicas)

É curioso que, apesar de tecnicamente superior, o JPEG 2000 acabou não substituindo o JPEG na web (por conta do custo computacional, de patentes na época, etc.). Ele venceu nos nichos onde a qualidade e a escalabilidade realmente importavam, como o cinema digital e as imagens médicas.

7.4 Demonstração final

Para finalizar, aplicamos o pipeline completo em uma foto real (uma foto minha!), convertida para escala de cinza e redimensionada para 512×512 , com quatro níveis de qualidade:

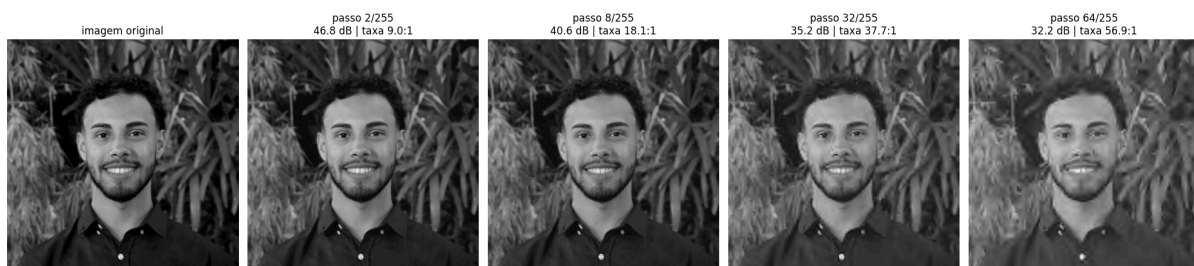


Imagem 11 - O pipeline JPEG 2000 simplificado aplicado a uma foto real, com passos de quantização de 2/255 até 64/255, mostrando o PSNR e a taxa de compressão de cada versão.

O comportamento é o mesmo observado na imagem de teste: nas taxas moderadas a compressão é visualmente transparente, e conforme apertamos o passo de quantização a imagem degrada de forma graciosa, borrando texturas finas antes de comprometer qualquer estrutura. Em nenhum momento aparecem blocos.

8. Conclusões

Depois de todo o desenvolvimento, podemos ver que a transformada de Wavelet é um exemplo de matemática que se traduz diretamente em tecnologia do dia a dia. Começamos com a operação mais boba possível (média e diferença de pares de números), mostramos que ela é uma mudança de base ortonormal, e terminamos com um compressor de imagens funcional no estilo do padrão usado no cinema digital. Foi a Álgebra Linear que mostra a ortogonalidade da matriz de Haar que nos deu inversa de graça, preservação de energia e o teorema de otimalidade da limiarização, e a análise multirresolução que organizou o espaço do sinal em somas diretas de subespaços por escala.

9. Referências

WIKIPEDIA CONTRIBUTORS. *Wavelet*. Disponível em: <<https://en.wikipedia.org/wiki/Wavelet>>.

WIKIPEDIA CONTRIBUTORS. *Haar wavelet*. Disponível em:

<https://en.wikipedia.org/wiki/Haar_wavelet>.

WIKIPEDIA CONTRIBUTORS. *Discrete wavelet transform*. Disponível em:

<https://en.wikipedia.org/wiki/Discrete_wavelet_transform>.

WIKIPEDIA CONTRIBUTORS. *Cohen–Daubechies–Feauveau wavelet*. Disponível em:

<https://en.wikipedia.org/wiki/Cohen%E2%80%93Daubechies%E2%80%93Feauveau_wavelet>.

WIKIPEDIA CONTRIBUTORS. *Lifting scheme*. Disponível em:

<https://en.wikipedia.org/wiki/Lifting_scheme>.

WIKIPEDIA CONTRIBUTORS. *JPEG 2000*. Disponível em:

<https://en.wikipedia.org/wiki/JPEG_2000>.

WIKIPEDIA CONTRIBUTORS. *Daubechies wavelet*. Disponível em:

<https://en.wikipedia.org/wiki/Daubechies_wavelet>.

WIKIPEDIA CONTRIBUTORS. *Multiresolution analysis*. Disponível em:

<https://en.wikipedia.org/wiki/Multiresolution_analysis>.

WIKIPEDIA CONTRIBUTORS. *Peak signal-to-noise ratio*. Disponível em:

<https://en.wikipedia.org/wiki/Peak_signal-to-noise_ratio>.

WIKIPEDIA CONTRIBUTORS. *Shannon's source coding theorem*. Disponível em:

<https://en.wikipedia.org/wiki/Shannon%27s_source_coding_theorem>.

JPEG.ORG. *Overview of JPEG 2000*. Disponível em: <<https://jpeg.org/jpeg2000/>>.